

# DTS103TC Coursework Report

## Algorithmic Solutions for E-Commerce Order Processing and Delivery

Jingyu Ye  
Student ID: 2470680  
DTS103TC

### Abstract

This report presents algorithmic solutions to four e-commerce order processing and delivery problems. The tasks cover maximum subarray computation, hash-table augmentation for top-product queries, package scheduling with dynamic programming and greedy methods, and minimum-cost grid traversal using Dijkstra's algorithm and 0-1 BFS.

## AI Usage Statement

Claude was used to support language polishing, code variable/function naming, and the refinement of code comments. The algorithmic ideas, solution design, implementation decisions, correctness arguments, and complexity analyses were reviewed and finalised by the author.

## Contents

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Task 1: Maximum Subarray Sum via Divide and Conquer</b>    | <b>2</b>  |
| 1.1      | Algorithm Design and Implementation . . . . .                 | 2         |
| 1.2      | Step-by-Step Execution Trace . . . . .                        | 3         |
| 1.3      | Recurrence Relation and Asymptotic Analysis . . . . .         | 5         |
| <b>2</b> | <b>Task 2: Pending Order Tracker with Top-Product Queries</b> | <b>8</b>  |
| 2.1      | Augmented Data Structure Design . . . . .                     | 8         |
| 2.2      | Average-Case Complexity Analysis . . . . .                    | 9         |
| 2.3      | Design Trade-Offs by Query Frequency . . . . .                | 9         |
| <b>3</b> | <b>Task 3: Package Delivery Scheduling</b>                    | <b>11</b> |
| 3.1      | Dynamic Programming Solution . . . . .                        | 11        |
| 3.2      | Greedy Approach and Matroid Justification . . . . .           | 13        |
| 3.3      | Generalisation to Variable Delivery Times . . . . .           | 15        |
| <b>4</b> | <b>Task 4: Minimum Delivery Time on a Grid</b>                | <b>17</b> |
| 4.1      | Dijkstra-Based Algorithm . . . . .                            | 17        |
| 4.2      | Cell Count vs. Total Travel Time . . . . .                    | 19        |
| 4.3      | Optimised Algorithm for $\{0,1\}$ Weights . . . . .           | 20        |

# 1 Task 1: Maximum Subarray Sum via Divide and Conquer

## 1.1 Algorithm Design and Implementation

### 1.1.1 Algorithm Design

The problem asks for the maximum sum over any contiguous subarray of `orders_change`, which is a variant of the classical *Maximum Subarray Problem*. A naive enumeration of all  $O(n^2)$  subarrays would require  $O(n^2)$  time (with prefix sums) or  $O(n^3)$  time (without). To improve on this, I adopt the divide-and-conquer paradigm introduced in Lecture 2, but with a refinement that reduces the combine step to constant time.

Given an array  $A[\text{low}..\text{high}]$ , the standard divide-and-conquer formulation splits the array at the midpoint and observes that the optimal subarray must lie entirely in the left half, entirely in the right half, or straddle the midpoint. The textbook approach handles the third case by scanning outward from the midpoint in linear time, yielding the recurrence  $T(n) = 2T(n/2) + \Theta(n)$  and a  $\Theta(n \log n)$  overall running time.

The key insight behind my implementation is that the linear-time crossing scan is redundant: if each recursive call returns enough auxiliary information about its subarray, the combine step can be performed in  $O(1)$  time. Specifically, each recursive call returns a 4-tuple:

- `total_sum` — the sum of every element in the segment.
- `max_prefix` — the maximum sum of any non-empty prefix of the segment.
- `max_suffix` — the maximum sum of any non-empty suffix of the segment.
- `max_subarray` — the maximum sum of any non-empty contiguous subarray inside the segment.

The combine step then computes the parent's 4-tuple from the two children's 4-tuples using only a constant number of additions and comparisons. This remains a genuine divide-and-conquer algorithm: the array is still split into two halves, each half is solved recursively, and the parent's answer is constructed from the recursive results — only the cost of the combine step has changed. As I show in Task 1-3, this design improves the running time to  $\Theta(n)$ , which is asymptotically optimal since every element must be examined at least once.

### 1.1.2 Python Implementation

```
1 from typing import List, Tuple
2
3 def max_order_increase(orders_change: List[int]) -> int:
4
5     if not orders_change:
6         return 0
7
8     def solve(low: int, high: int) -> Tuple[int, int, int, int]:
9
10        if low == high:
11            value = orders_change[low]
12            return value, value, value, value
13        mid = (low + high) // 2
14
15        left_total, left_prefix, left_suffix, left_best = solve(low, mid)
16        right_total, right_prefix, right_suffix, right_best = solve(mid + 1,
17        high)
18
19        total_sum = left_total + right_total
20
```

```

21     max_prefix = max(left_prefix, left_total + right_prefix)
22
23     max_suffix = max(right_suffix, right_total + left_suffix)
24
25     crossing_best = left_suffix + right_prefix
26
27     max_subarray = max(left_best, right_best, crossing_best)
28
29     return total_sum, max_prefix, max_suffix, max_subarray
30
31 return solve(0, len(orders_change) - 1)[3]

```

Listing 1: Divide-and-conquer maximum subarray algorithm.

### 1.1.3 Correctness Justification

I argue correctness by induction on the size of the segment.

**Base case** (`low == high`). The segment contains a single element  $v$ . The only non-empty prefix, suffix, and subarray is  $v$  itself, and the total sum is also  $v$ . The returned tuple  $(v, v, v, v)$  is therefore correct.

**Inductive step.** Assume the algorithm correctly returns the 4-tuple for both halves of a segment of size  $n > 1$ . I verify each of the four returned values for the combined segment:

- `total_sum` is trivially the sum of the two halves' totals.
- The best prefix of the combined segment either ends inside the left half — in which case it equals the left half's best prefix — or it extends through the entire left half into the right half, in which case it equals `left_total + right_prefix`. Taking the maximum covers both cases.
- The best suffix follows by symmetric reasoning.
- Any non-empty contiguous subarray of the combined segment lies either entirely in the left half, entirely in the right half, or it crosses the midpoint. A crossing subarray necessarily ends somewhere in the left half and begins somewhere in the right half; its sum is therefore (some suffix sum of left) + (some prefix sum of right), which is maximised by `left_suffix + right_prefix`. Taking the maximum of the three cases gives the best subarray of the combined segment.

By induction, the algorithm returns the correct maximum subarray sum for any input of size at least 1. The wrapper function handles the empty-input edge case explicitly.

### 1.1.4 Efficiency Discussion

As I derive formally in Task 1-3, the recurrence  $T(n) = 2T(n/2) + O(1)$  solves to  $T(n) = \Theta(n)$ , which is asymptotically optimal — any correct algorithm must examine every element at least once. The auxiliary space is  $O(\log n)$  for the recursion stack, since each recursive call stores only a constant-size tuple. By comparison, the textbook divide-and-conquer formulation that scans the crossing subarray achieves only  $\Theta(n \log n)$  time.

## 1.2 Step-by-Step Execution Trace

I trace the algorithm on the input `orders_change = [4, -2, 3, -1, 5, -3]` (indices 0 through 5). Each node of the recursion tree below is annotated with the segment it solves and the 4-tuple it returns, in the form  $(total, prefix, suffix, best)$ .

### 1.2.1 Recursion Tree

```

1      solve(0, 5)  -> (6, 9, 6, 9)
2      [4, -2, 3, -1, 5, -3]
3      mid = 2
4      +-----+-----+
5      |               |
6      solve(0, 2)      solve(3, 5)
7      [4, -2, 3]        [-1, 5, -3]
8      ->(5, 5, 5, 5)    ->(1, 4, 2, 5)
9      mid = 1          mid = 4
10     +-----+-----+ +-----+-----+
11     |               |               |               |
12     solve(0, 1)    solve(2, 2)    solve(3, 4)    solve(5, 5)
13     [4, -2]        [3]            [-1, 5]        [-3]
14     ->(2, 4, 2, 4)  ->(3, 3, 3, 3) ->(4, 4, 5, 5)  ->(-3, -3, -3, -3)
15     mid = 0        mid = 3
16     +-----+-----+ +-----+-----+
17     |               |               |               |
18     solve(0, 0)    solve(1, 1)    solve(3, 3)    solve(4, 4)
19     [4]            [-2]            [-1]            [5]
20     ->(4, 4, 4, 4) ->(-2, -2, -2, -2) ->(-1, -1, -1, -1) ->(5, 5, 5, 5)
21

```

### 1.2.2 Detailed Combine Steps

I show each non-leaf combine step explicitly. All four returned values are computed from the children's tuples using only the closed-form formulas given in Task 1-1.

**Node solve(0, 1) on [4, -2].** Children return (4, 4, 4, 4) and (-2, -2, -2, -2).

```

1 total_sum      = 4 + (-2)                = 2
2 max_prefix     = max(4, 4 + (-2))         = max(4, 2)    = 4
3 max_suffix     = max(-2, -2 + 4)          = max(-2, 2)    = 2
4 crossing_best  = 4 + (-2)                = 2
5 max_subarray   = max(4, -2, 2)            = 4
6
7 Returns (2, 4, 2, 4).

```

**Node solve(0, 2) on [4, -2, 3].** Children return (2, 4, 2, 4) and (3, 3, 3, 3).

```

1 total_sum      = 2 + 3                    = 5
2 max_prefix     = max(4, 2 + 3)            = max(4, 5)    = 5
3 max_suffix     = max(3, 3 + 2)            = max(3, 5)    = 5
4 crossing_best  = 2 + 3                    = 5
5 max_subarray   = max(4, 3, 5)             = 5
6
7 Returns (5, 5, 5, 5).

```

The best subarray of the left half is [4, -2, 3] itself, with sum 5.

**Node solve(3, 4) on [-1, 5].** Children return (-1, -1, -1, -1) and (5, 5, 5, 5).

```

1 total_sum      = -1 + 5                   = 4
2 max_prefix     = max(-1, -1 + 5)          = max(-1, 4)    = 4
3 max_suffix     = max(5, 5 + (-1))         = max(5, 4)    = 5
4 crossing_best  = -1 + 5                   = 4
5 max_subarray   = max(-1, 5, 4)            = 5
6
7 Returns (4, 4, 5, 5).

```

**Node solve(3, 5) on [-1, 5, -3].** Children return (4, 4, 5, 5) and (-3, -3, -3, -3).

Computing `max_suffix` carefully: the formula is  $\max(\text{right\_suffix}, \text{right\_total} + \text{left\_suffix})$ . Here  $\text{right\_suffix} = -3$  and  $\text{right\_total} + \text{left\_suffix} = -3 + 5 = 2$ .

```

1 total_sum      = 4 + (-3)                = 1
2 max_prefix     = max(4, 4 + (-3))         = max(4, 1)     = 4
3 max_suffix     = max(-3, -3 + 5)          = max(-3, 2)     = 2
4 crossing_best  = 5 + (-3)                = 2
5 max_subarray   = max(5, -3, 2)           = 5
6
7 Returns (1, 4, 2, 5).
```

The best subarray of the right half is the singleton [5], with sum 5.

**Root node solve(0, 5).** Children return (5, 5, 5, 5) and (1, 4, 2, 5).

```

1 total_sum      = 5 + 1                    = 6
2 max_prefix     = max(5, 5 + 4)            = max(5, 9)     = 9
3 max_suffix     = max(2, 1 + 5)            = max(2, 6)     = 6
4 crossing_best  = 5 + 4                    = 9
5 max_subarray   = max(5, 5, 9)             = 9
6
7 Returns (6, 9, 6, 9).
```

### 1.2.3 Result

The algorithm returns [9]. This corresponds to the contiguous subarray [4, -2, 3, -1, 5] (indices 0 through 4), whose sum is  $4 + (-2) + 3 + (-1) + 5 = 9$ . Note that this optimal subarray straddles the top-level midpoint, and is recovered specifically by the crossing case at the root — neither half alone contains it. This illustrates why all three candidates (left, right, and crossing) must be considered at every combine step.

## 1.3 Recurrence Relation and Asymptotic Analysis

### 1.3.1 Recurrence Relation

Let  $T(n)$  denote the worst-case running time of `solve(low, high)` on a segment of size  $n$ . The algorithm performs the following work at each recursive call:

1. **Base case** ( $n = 1$ ). The function constructs and returns a 4-tuple from a single element in constant time. Hence  $T(1) = \Theta(1)$ .
2. **Recursive case** ( $n > 1$ ). The algorithm makes two recursive calls on subproblems of size  $\lfloor n/2 \rfloor$  and  $\lceil n/2 \rceil$ , and the combine step performs a fixed number of additions and `max` comparisons on the eight values returned by the two children. This is constant work — crucially, **independent of  $n$**  — because each recursive call returns only a 4-tuple, regardless of the size of the segment it covers.

The recurrence is therefore:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(1) & \text{if } n > 1 \end{cases}$$

Assuming for analysis that  $n$  is a power of 2 (which affects only constant factors), and writing the constant non-recursive work as  $c$ :

$$T(n) = 2T\left(\frac{n}{2}\right) + c$$

I solve this recurrence using two independent methods.

### 1.3.2 Method 1: Recursion-Tree Method

The recursion-tree method unrolls the recurrence into a tree, counts the work performed at each level, and sums across levels.

**Tree structure.** The root represents the original problem of size  $n$  and contributes  $c$  units of non-recursive work. It has two children, each representing a subproblem of size  $n/2$ . Each of those has two children of size  $n/4$ , and so on. The tree continues until subproblems reach size 1.

|   |                     |                                                                   |                       |
|---|---------------------|-------------------------------------------------------------------|-----------------------|
| 1 | Level 0:            | $c$                                                               | -> work = $c$         |
| 2 |                     | $\begin{array}{c} / \quad \backslash \end{array}$                 |                       |
| 3 | Level 1:            | $\begin{array}{cc} c & c \end{array}$                             | -> work = $2c$        |
| 4 |                     | $\begin{array}{cccc} / & \backslash & / & \backslash \end{array}$ |                       |
| 5 | Level 2:            | $\begin{array}{cccc} c & c & c & c \end{array}$                   | -> work = $4c$        |
| 6 |                     | $\dots$                                                           |                       |
| 7 | Level $k$ :         | $2^k$ nodes, each contributing $c$                                | -> work = $(2^k) * c$ |
| 8 |                     | $\dots$                                                           |                       |
| 9 | Level $\log_2(n)$ : | $n$ nodes, each $\Theta(1)$ base-case                             | -> work = $\Theta(n)$ |

**Number of levels.** The subproblem size at level  $k$  is  $n/2^k$ . The base case is reached when  $n/2^k = 1$ , i.e. when  $k = \log_2 n$ . The tree therefore has  $\log_2 n + 1$  levels (levels 0 through  $\log_2 n$ ).

**Work per level.** At level  $k$ , there are  $2^k$  nodes, and each contributes  $c$  units of work in the recursive case (or  $\Theta(1)$  in the base case). The total work at level  $k$  is:

$$2^k \cdot c$$

This **doubles at each level**, unlike the merge-sort recurrence ( $T(n) = 2T(n/2) + cn$ ) where the work stays constant per level. This is the structural reason why my algorithm achieves  $\Theta(n)$  rather than  $\Theta(n \log n)$ .

**Total work.** Summing across all levels:

$$\begin{aligned}
 T(n) &= c \cdot (1 + 2 + 4 + \dots + 2^{\log_2 n - 1}) + \Theta(n) \\
 &= c \cdot (2^{\log_2 n} - 1) + \Theta(n) \quad [\text{geometric series}] \\
 &= c \cdot (n - 1) + \Theta(n) \\
 &= \Theta(n)
 \end{aligned}$$

Since every element of the input must be examined at least once,  $T(n) = \Omega(n)$  is also a trivial lower bound. Combining:

$$\boxed{T(n) = \Theta(n)}$$

### 1.3.3 Method 2: Master Theorem

The recurrence  $T(n) = 2T(n/2) + c$  matches the standard form of the Master Theorem:

$$T(n) = aT(n/b) + f(n), \quad \text{where } a = 2, \ b = 2, \ f(n) = \Theta(1)$$

**Step 1: Compute the watershed function.**

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

**Step 2: Compare  $f(n)$  against  $n^{\log_b a}$ .** I have  $f(n) = \Theta(1)$  and  $n^{\log_b a} = n$ . To apply Case 1, I need to show that  $f(n) = O(n^{\log_b a - \varepsilon})$  for some constant  $\varepsilon > 0$ . Choosing  $\varepsilon = 1$ :

$$f(n) = \Theta(1) = O(n^{1-1}) = O(n^0) = O(1) \quad \checkmark$$

This satisfies the condition of **Case 1** of the Master Theorem.

**Step 3: Apply Case 1.** Case 1 states: if  $f(n) = O(n^{\log_b a - \varepsilon})$  for some  $\varepsilon > 0$ , then:

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$$

### 1.3.4 Cross-Verification and Space Complexity

Both methods independently yield the same asymptotic bound:

$$\boxed{T(n) = \Theta(n)}$$

The recursion-tree analysis provides the structural intuition — the work at level  $k$  is  $2^k \cdot c$ , which grows geometrically and is dominated by the leaf level, giving a total of  $\Theta(n)$ . The Master Theorem confirms this algebraically by placing the recurrence in Case 1, where the recursive calls (which proliferate exponentially) dominate the non-recursive work (which is constant per call).

The algorithm passes only index boundaries between recursive calls and never copies subarrays. Each active call stores a constant-size 4-tuple. Since the recursion tree has depth  $\log_2 n$ , the auxiliary space complexity is  $O(\log n)$ .

| Metric                             | Result                       |
|------------------------------------|------------------------------|
| Recurrence                         | $T(n) = 2T(n/2) + \Theta(1)$ |
| Time complexity                    | $\Theta(n)$                  |
| Auxiliary space                    | $O(\log n)$                  |
| Output for $[4, -2, 3, -1, 5, -3]$ | 9                            |

This is asymptotically optimal: any algorithm correctly solving the problem must examine every element at least once, giving an  $\Omega(n)$  lower bound, which my algorithm matches.

## 2 Task 2: Pending Order Tracker with Top-Product Queries

### 2.1 Augmented Data Structure Design

**Design rationale.** The original `ProductTracker` uses a hash table that supports `add_order`, `process_order`, and `get_pending` in expected  $O(1)$  time. To support `top_product` efficiently, a naïve scan over the dictionary would cost  $O(n)$  per query, which becomes a bottleneck when the operation is invoked frequently. A better approach is to augment the hash table with an auxiliary **max-heap** that stores  $(-pending, product\_id)$  pairs. Since Python's `heapq` is a min-heap, negating the pending count converts it into a max-heap.

A subtle issue is that `add_order` and `process_order` mutate the pending count of a product that may already appear in the heap. Rebuilding the heap on every update would cost  $O(n)$ , which defeats the purpose of the optimisation. Instead, we adopt **lazy deletion**: each update pushes a fresh entry onto the heap without removing the stale one, and `top_product` discards stale entries from the top of the heap until it finds one whose stored count agrees with the authoritative value in the hash table.

To keep the dictionary compact and to ensure `top_product` correctly returns `None` when no product has positive pending orders, products whose pending count falls to zero are removed from the dictionary entirely. The heap check `self.pending.get(pid, 0) == count` then naturally filters out such stale entries.

```
1 import heapq
2
3
4 class ProductTracker:
5     def __init__(self):
6         self.pending = {}      # product_id -> current pending orders
7         self.heap = []        # stores (-pending, product_id); may contain stale
                                # entries
8
9     def add_order(self, product_id, quantity):
10        if quantity < 0:
11            raise ValueError("quantity must be non-negative")
12        if quantity == 0:
13            return
14
15        new_count = self.pending.get(product_id, 0) + quantity
16        self.pending[product_id] = new_count
17        heapq.heappush(self.heap, (-new_count, product_id))
18
19    def process_order(self, product_id, quantity):
20        if quantity < 0:
21            raise ValueError("quantity must be non-negative")
22
23        current = self.pending.get(product_id, 0)
24        new_count = max(0, current - quantity)
25
26        if new_count == 0:
27            self.pending.pop(product_id, None)
28        else:
29            self.pending[product_id] = new_count
30            heapq.heappush(self.heap, (-new_count, product_id))
31
32    def get_pending(self, product_id):
33        return self.pending.get(product_id, 0)
34
35    def top_product(self):
36        while self.heap:
37            neg_count, pid = self.heap[0]    # peek without popping
```

```

38         count = -neg_count
39
40         if count > 0 and self.pending.get(pid, 0) == count:
41             return pid
42         heapq.heappop(self.heap)
43     return None # no products with positive pending orders

```

Listing 2: Heap-augmented product tracker with lazy deletion.

**Correctness invariant.** After every successful update to a product with positive pending orders, the heap contains at least one entry representing that product’s current count. Stale entries may persist, but they are filtered at query time. When `top_product` is called, the heap root has the largest stored count among all heap entries. If the root agrees with the dictionary, it must represent a product with the largest current pending count: if any other product had a larger current count, its most recent update would have pushed an entry at least as large as the current root, contradicting the heap property. If the root disagrees with the dictionary, it is stale and is discarded. The loop therefore returns a valid maximum, or `None` when no positive pending orders remain.

## 2.2 Average-Case Complexity Analysis

Let  $n$  denote the number of distinct products currently tracked (i.e., with positive pending orders).

| Operation                  | Average-case complexity | Justification                                                         |
|----------------------------|-------------------------|-----------------------------------------------------------------------|
| <code>__init__</code>      | $O(1)$                  | Initialises an empty dictionary and an empty list.                    |
| <code>add_order</code>     | $O(\log n)$             | Hash-table update (expected $O(1)$ ) plus heap push ( $O(\log n)$ ).  |
| <code>process_order</code> | $O(\log n)$             | Hash-table update or deletion plus heap push ( $O(\log n)$ ).         |
| <code>get_pending</code>   | $O(1)$                  | A single hash-table lookup, unchanged from the original.              |
| <code>top_product</code>   | $O(\log n)$ amortised   | Stale-entry cleanups are charged to the operations that created them. |

**Amortised reasoning for `top_product`.** Consider any sequence of  $k$  mixed operations. Each `add_order` and `process_order` performs at most one heap push, so the total number of heap entries created across the sequence is at most  $k$ . Consequently, the total number of pops performed by `top_product` calls (including stale-entry cleanups) is also at most  $k$ . Each push and each pop costs  $O(\log n)$ , giving total work  $O(k \log n)$  and an amortised  $O(\log n)$  per operation. When the top of the heap is already valid, the call returns in  $O(1)$ .

**Space complexity.** The data structure uses  $O(n + s)$  space, where  $s$  is the number of stale heap entries that have not yet been discarded. In a pathological workload where many updates accumulate before any `top_product` call,  $s$  could grow large; if monitored, the heap can be rebuilt from the dictionary in  $O(n)$  time to evict all stale entries at once.

## 2.3 Design Trade-Offs by Query Frequency

The choice of data structure depends on the workload, and this question illustrates a classic update–query trade-off.

### 2.3.1 Case 1: Rare top\_product Queries

If `top_product` is invoked infrequently, maintaining an auxiliary heap is wasteful: every `add_order` and `process_order` would pay an  $O(\log n)$  heap push whose result is almost never consulted. In this regime, the **plain hash-table design** is preferable. Keep `add_order`, `process_order`, and `get_pending` at  $O(1)$  average, and implement `top_product` as a one-off linear scan:

```
1 def top_product(self):
2     if not self.pending:
3         return None
4     return max(self.pending, key=self.pending.get)
```

This costs  $O(n)$  per query, but since queries are rare, the cost is amortised over a long stream of cheap  $O(1)$  updates. The implementation is simpler, has lower constant factors, and avoids the extra memory overhead of the heap.

### 2.3.2 Case 2: Frequent top\_product Queries

When `top_product` is on the hot path, the  $O(n)$  linear scan becomes a bottleneck and an auxiliary indexed structure pays off. Two designs are appropriate.

The first is the **hash table plus max-heap with lazy deletion** implemented in Task 2-1. Updates cost  $O(\log n)$ , queries are answered in  $O(\log n)$  amortised time (often  $O(1)$  when no cleanup is needed), and the implementation is simple to layer on top of the existing dictionary. The only extra memory is the  $O(n + s)$  for the heap, including stale entries.

The second, more sophisticated option is a **hash table plus a balanced binary search tree** keyed by  $(pending\_count, product\_id)$  — for example, a red-black tree or a treap. This supports updates, deletions, and maximum retrieval all in worst-case  $O(\log n)$ , without the bookkeeping required by lazy deletion. It is the right choice when deterministic worst-case bounds matter (e.g., real-time systems).

A Fibonacci heap is sometimes cited in this context, but its theoretical advantage ( $O(1)$  amortised decrease-key) does not apply to our lazy-deletion design, since we never call decrease-key — we simply push fresh entries. Its large constant factors rarely justify it in practice.

### 2.3.3 General Principle

Augmenting a primary structure with an auxiliary indexed structure only pays off when the secondary query is **hot enough** to amortise the per-update maintenance cost. Identifying the break-even point requires understanding the actual call-frequency profile of the system. As a rough rule, if the ratio of `top_product` calls to updates is much smaller than  $\log n/n$ , the linear-scan design wins; otherwise, the heap-augmented design does.

| Usage pattern                    | Recommended design           | Updates           | top_product           |
|----------------------------------|------------------------------|-------------------|-----------------------|
| top_product rare                 | Hash table only              | $O(1)$ avg        | $O(n)$                |
| top_product frequent             | Hash table + max-heap (lazy) | $O(\log n)$       | $O(\log n)$ amortised |
| Both frequent, worst-case bounds | Hash table + balanced BST    | $O(\log n)$ worst | $O(\log n)$ worst     |

### 3 Task 3: Package Delivery Scheduling

#### 3.1 Dynamic Programming Solution

##### 3.1.1 Problem Analysis

We are given  $n$  packages, each with a deadline  $d_i$  and a penalty  $p_i$ . Each package requires exactly one unit of time to deliver, and delivery begins at time slot 1. A package contributes its penalty to the total cost if and only if it is delivered after its deadline. The objective is to minimise the total penalty incurred.

**Key observation.** Minimising the penalty paid is equivalent to *maximising* the total penalty *saved* by selecting a feasible set of packages that can be delivered on time. Let  $S$  denote the set of on-time packages; then

$$\text{minimum penalty} = \sum_{i=1}^n p_i - \max_{S \text{ feasible}} \sum_{i \in S} p_i.$$

A subset  $S$  is *feasible* if its packages can be scheduled so that every package meets its deadline. A classical result on unit-time scheduling states that  $S$  is feasible if and only if, after sorting  $S$  by deadline in non-decreasing order, the  $k$ -th package in this ordering has deadline at least  $k$ . Intuitively, the earliest-deadline-first ordering is always optimal for any feasible set, so the only thing that matters is whether each on-time package can be assigned to a distinct time slot at or before its deadline.

##### 3.1.2 Dynamic Programming Formulation

After sorting the packages by deadline in non-decreasing order, let

$$f[i][t] = \text{max. penalty saved using the first } i \text{ packages and delivering exactly } t \text{ on time.}$$

For each package  $i$  with deadline  $d_i$  and penalty  $p_i$ , there are two choices:

- **Skip package  $i$  (treat as late):**  $f[i][t] = f[i-1][t]$ .
- **Schedule package  $i$  on time,** allowed only when  $t \leq d_i$ :  $f[i][t] = f[i-1][t-1] + p_i$ .

The recurrence is therefore

$$f[i][t] = \begin{cases} \max(f[i-1][t], f[i-1][t-1] + p_i) & \text{if } 1 \leq t \leq d_i, \\ f[i-1][t] & \text{if } t > d_i, \end{cases}$$

with base case  $f[0][0] = 0$  and  $f[0][t] = -\infty$  for  $t > 0$ . The maximum penalty saved is  $\max_t f[n][t]$ , and the answer is  $\sum_i p_i - \max_t f[n][t]$ .

##### 3.1.3 Memory Optimisation

Since  $f[i][\cdot]$  depends only on  $f[i-1][\cdot]$ , we compress the table into a single one-dimensional array of length  $D+1$ , where  $D = \min(n, \max_i d_i)$ . Iterating  $t$  from high to low — the standard 0-1 knapsack space-optimisation technique — ensures that each package is used at most once, because the value  $f[i-1][t-1]$  is read before being overwritten.

### 3.1.4 Python Implementation

```

1 from typing import List
2
3 def min_penalty(deadline: List[int], penalty: List[int]) -> int:
4     if len(deadline) != len(penalty):
5         raise ValueError("deadline and penalty must have the same length.")
6
7     n = len(deadline)
8     if n == 0:
9         return 0
10
11     total_penalty = sum(penalty)
12     max_deadline = max(deadline)
13     if max_deadline <= 0:
14         return total_penalty
15
16     D = min(n, max_deadline)
17
18     packages = sorted(zip(deadline, penalty), key=lambda x: x[0])
19
20     NEG_INF = float('-inf')
21     dp = [NEG_INF] * (D + 1)
22     dp[0] = 0 # zero on-time packages saves zero penalty
23
24     for d_i, p_i in packages:
25         upper = min(d_i, D)
26         for t in range(upper, 0, -1):
27             if dp[t - 1] != NEG_INF:
28                 candidate = dp[t - 1] + p_i
29                 if candidate > dp[t]:
30                     dp[t] = candidate
31
32     max_saved = max(v for v in dp if v != NEG_INF)
33     return total_penalty - max_saved

```

Listing 3: Dynamic programming solution for unit-time scheduling.

### 3.1.5 Correctness Verification

I trace the algorithm on the brief's example to confirm that the implementation produces the expected output of 30. The packages are first sorted by deadline (ties broken arbitrarily), giving the sequence

$$(d=1, p=10), (d=1, p=20), (d=2, p=5), (d=2, p=25), (d=2, p=15).$$

With  $n = 5$  and  $\max_i d_i = 2$ , we take  $D = \min(5, 2) = 2$ , so the DP array has length 3, initialised as  $\text{dp} = [0, -\infty, -\infty]$ .

The state  $\text{dp}[t]$  represents the maximum penalty saved by delivering exactly  $t$  packages on time among those processed so far. Each row below shows the array after fully processing one package, with the iteration of  $t$  running from  $\text{upper} = \min(d_i, D)$  downward.

| Step | Package $(d, p)$ | upper | Updates                                                                                               | dp after step           |
|------|------------------|-------|-------------------------------------------------------------------------------------------------------|-------------------------|
| 0    | — (initial)      | —     | —                                                                                                     | $[0, -\infty, -\infty]$ |
| 1    | (1, 10)          | 1     | $\text{dp}[1] \leftarrow \max(-\infty, 0 + 10) = 10$                                                  | $[0, 10, -\infty]$      |
| 2    | (1, 20)          | 1     | $\text{dp}[1] \leftarrow \max(10, 0 + 20) = 20$                                                       | $[0, 20, -\infty]$      |
| 3    | (2, 5)           | 2     | $\text{dp}[2] \leftarrow \max(-\infty, 20 + 5) = 25$ ; $\text{dp}[1] \leftarrow \max(20, 0 + 5) = 20$ | $[0, 20, 25]$           |
| 4    | (2, 25)          | 2     | $\text{dp}[2] \leftarrow \max(25, 20 + 25) = 45$ ; $\text{dp}[1] \leftarrow \max(20, 0 + 25) = 25$    | $[0, 25, 45]$           |
| 5    | (2, 15)          | 2     | $\text{dp}[2] \leftarrow \max(45, 25 + 15) = 45$ ; $\text{dp}[1] \leftarrow \max(25, 0 + 15) = 25$    | $[0, 25, 45]$           |

The maximum saved penalty is  $\max(\text{dp}) = 45$ , attained at  $t = 2$ . This corresponds to selecting package 2 (penalty 20, scheduled at slot 1) and package 4 (penalty 25, scheduled at slot 2). The minimum total penalty is therefore

$$\sum_i p_i - \max_t \text{dp}[t] = 75 - 45 = \boxed{30},$$

matching the expected output. Note in particular how the downward iteration of  $t$  at step 4 prevents package 4 from being double-counted: the update  $\text{dp}[2] \leftarrow \text{dp}[1] + 25 = 45$  reads the pre-update value  $\text{dp}[1] = 20$ , before that slot itself is overwritten to 25.

### 3.1.6 Complexity Analysis

- **Time complexity.** Sorting costs  $O(n \log n)$ . The DP update considers at most  $D = \min(n, \max_i d_i)$  slots for each of the  $n$  packages, contributing  $O(nD)$ . The overall time complexity is  $O(n \log n + nD)$ , which is  $O(n^2)$  in the worst case when  $D = \Theta(n)$ .
- **Space complexity.** The algorithm uses a single one-dimensional DP array of size  $D + 1$ , giving  $O(D)$  auxiliary space — a substantial improvement over the naïve  $O(nD)$  two-dimensional table.

## 3.2 Greedy Approach and Matroid Justification

**Yes.** This problem admits an optimal greedy solution. In fact, it is a classical example of scheduling on a unit-time task system, which forms a *weighted matroid*. A foundational theorem of combinatorial optimisation states that the greedy algorithm — repeatedly adding the heaviest element that preserves independence — produces a maximum-weight independent set on any weighted matroid. Since maximising the on-time penalty corresponds exactly to finding such a set, the greedy approach is guaranteed to be optimal.

### 3.2.1 Greedy Strategy

A clean implementation uses a min-heap:

1. Sort the packages by non-decreasing deadline.
2. Maintain a min-heap containing the penalties of packages currently selected to be delivered on time.
3. For each package in sorted order:
  - Insert its penalty into the heap.
  - If the heap size exceeds the current deadline, extract the smallest penalty (the package least worth saving).
4. At the end, the heap contains the saved packages; the answer is total penalty  $- \text{sum}(\text{heap})$ .

### 3.2.2 Why This Works

The argument has two equivalent justifications.

**Exchange argument.** When considering all packages with deadline at most  $d$ , at most  $d$  of them can be on time. If more than  $d$  have been provisionally selected, at least one must be late. To maximise the saved penalty, the package made late should be the one with the smallest penalty among those currently selected — removing it causes the least possible loss. This local choice never excludes an optimal solution: if some optimal schedule kept the smallest-penalty package and dropped a larger one, we could swap them without reducing the total saved penalty, so an optimal solution agreeing with the greedy choice always exists.

**Matroid-theoretic argument.** The independent sets (feasible on-time subsets) of this scheduling problem are closed under subsets and satisfy the matroid exchange property. The greedy algorithm on a weighted matroid is provably optimal — this is a standard result.

### 3.2.3 Python Implementation

```

1 import heapq
2 from typing import List
3
4 def min_penalty_greedy(deadline: List[int], penalty: List[int]) -> int:
5     if len(deadline) != len(penalty):
6         raise ValueError("deadline and penalty must have the same length.")
7
8     packages = sorted(zip(deadline, penalty), key=lambda x: x[0])
9     total_penalty = sum(penalty)
10
11     selected = [] # min-heap of penalties of currently on-time packages
12     for d, p in packages:
13         if d <= 0:
14             continue
15         heapq.heappush(selected, p)
16         if len(selected) > d:
17             heapq.heappop(selected) # drop the smallest penalty
18
19     return total_penalty - sum(selected)

```

Listing 4: Greedy solution using a min-heap.

### 3.2.4 Verification on the Example

Processing packages sorted by deadline:

| Step | Package ( $d, p$ ) | Heap after push | Heap size > $d$ ? | Heap after pop |
|------|--------------------|-----------------|-------------------|----------------|
| 1    | (1, 10)            | [10]            | No                | [10]           |
| 2    | (1, 20)            | [10, 20]        | Yes               | [20]           |
| 3    | (2, 5)             | [5, 20]         | No                | [5, 20]        |
| 4    | (2, 25)            | [5, 20, 25]     | Yes               | [20, 25]       |
| 5    | (2, 15)            | [15, 25, 20]    | Yes               | [20, 25]       |

Saved penalty =  $20 + 25 = 45$ ; minimum total penalty =  $75 - 45 = 30$ . ✓

### 3.2.5 Complexity

Sorting takes  $O(n \log n)$ . Each package triggers at most one heap push and one heap pop, each costing  $O(\log n)$ . The overall time complexity is  $O(n \log n)$ , and the space complexity is  $O(n)$  for the heap. This is asymptotically faster than the DP solution.

### 3.3 Generalisation to Variable Delivery Times

Suppose each package  $i$  now requires  $t_i$  units of time to deliver (still indivisible), with deadline  $d_i$  meaning package  $i$  must *finish* by time  $d_i$ . The problem becomes substantially harder, for two reasons.

**Feasibility is no longer characterised by a simple ranking condition.** A feasible on-time set  $S$  must now satisfy  $\sum_{j \in S, d_j \leq d_i} t_j \leq d_i$  for every distinct deadline  $d_i$ .

**The matroid structure breaks down.** Independent sets no longer form a matroid once tasks have variable durations, so the “sort by penalty, place at the latest slot” greedy strategy can be arbitrarily suboptimal. A polynomial-time greedy is therefore unlikely to exist.

#### 3.3.1 Modified DP Formulation

We again maximise the total saved penalty and subtract from the total. Sort packages by deadline as before. Let  $D = \max_i d_i$ , and redefine

$dp[i][t]$  = max. saved penalty using a subset of the first  $i$  packages with total delivery time exactly  $t$ .

For each package  $i$  with delivery time  $t_i$ , deadline  $d_i$ , and penalty  $p_i$ :

$$dp[i][t] = \begin{cases} \max(dp[i-1][t], dp[i-1][t-t_i] + p_i) & \text{if } t_i \leq t \leq d_i, \\ dp[i-1][t] & \text{otherwise.} \end{cases}$$

The constraint  $t \leq d_i$  enforces that package  $i$ , when scheduled on time, finishes by its deadline: because packages are processed in deadline order, the cumulative on-time duration  $t$  after including package  $i$  is exactly its finishing time. The minimum penalty is

$$\min \text{penalty} = \sum_i p_i - \max_{0 \leq t \leq D} dp[n][t].$$

As in Task 3-1, the two-dimensional table compresses to a one-dimensional array by iterating  $t$  from high to low.

#### 3.3.2 Python Implementation

```
1 from typing import List
2
3 def min_penalty_variable_time(deadline: List[int],
4                               penalty: List[int],
5                               delivery_time: List[int]) -> int:
6
7     if not (len(deadline) == len(penalty) == len(delivery_time)):
8         raise ValueError("inputs must all have the same length.")
9
10    n = len(deadline)
11    if n == 0:
12        return 0
13
14    total_penalty = sum(penalty)
15    max_deadline = max(deadline)
16    if max_deadline <= 0:
17        return total_penalty
18
19    D = min(sum(delivery_time), max_deadline)
20
```

```

21 packages = sorted(zip(deadline, penalty, delivery_time), key=lambda x: x
22 [0])
23 dp = [0] * (D + 1)
24
25 for d, p, c in packages:
26     if c <= 0:
27         continue
28     latest_finish_time = min(d, D)
29     for t in range(latest_finish_time, c - 1, -1):
30         if dp[t - c] + p > dp[t]:
31             dp[t] = dp[t - c] + p
32
33 max_saved = max(dp)
34 return total_penalty - max_saved

```

Listing 5: DP solution for variable-duration scheduling.

### 3.3.3 Correctness and Complexity

After sorting by deadline, any selected subset can be processed in earliest-deadline-first order. The state  $dp[t]$  stores the best saved penalty among feasible selections of total delivery time exactly  $t$ . When adding a package with delivery time  $c$  and deadline  $d$ , the package can only be accepted if it finishes at some time  $t \leq d$ ; the previously selected packages then occupy  $t - c$  units of time. The transition  $dp[t] = \max(dp[t], dp[t - c] + p)$  thus correctly represents the choice of rejecting the package or accepting it as an on-time delivery.

This is a direct generalisation of Task 3-1. In the original problem each package consumes one time slot, so the DP indexed only the *count* of on-time packages; here each package consumes  $t_i$  units, so the DP must index the *cumulative time* — making the problem structurally identical to 0-1 knapsack with deadline-induced upper bounds.

- **Time complexity:**  $O(n \cdot D)$ , where  $D = \min(\sum_i t_i, \max_i d_i)$ . Each of the  $n$  packages scans up to  $D$  time values. Sorting contributes only  $O(n \log n)$ , which is dominated whenever  $D \geq \log n$ .
- **Space complexity:**  $O(D)$  using the rolling-array optimisation, or  $O(nD)$  for the full table.

This is **pseudo-polynomial** in the input size, mirroring the situation with 0-1 knapsack — the running time depends on the numerical bound  $D$  rather than only on the number of packages. In fact, the variable-duration version is **NP-hard in the strong sense** when  $D$  can be exponentially large in the input size, so a fully polynomial-time exact algorithm is unlikely to exist. The greedy matroid argument from Task 3-2 also fails here, because variable-duration scheduling does not preserve the matroid exchange property; consequently no simple greedy substitute is known.

## 4 Task 4: Minimum Delivery Time on a Grid

### 4.1 Dijkstra-Based Algorithm

#### 4.1.1 Algorithm Design

The problem asks for the minimum total travel time from the top-left cell  $(0, 0)$  to the bottom-right cell  $(m - 1, n - 1)$  of an  $m \times n$  grid, where  $\text{grid}[i][j] \geq 0$  denotes the travel time of traversing cell  $(i, j)$ , and the courier may move in four directions (up, down, left, right).

We model the problem as a single-source shortest path problem on an implicit weighted graph  $G = (V, E)$ :

- $V = \{(i, j) : 0 \leq i < m, 0 \leq j < n\}$ , with  $|V| = mn$ .
- For each pair of orthogonally adjacent cells  $(i, j)$  and  $(i', j')$ , a directed edge of weight  $\text{grid}[i'][j']$  (the cost of *entering* the neighbour). Thus  $|E| = O(mn)$ .
- The source is  $(0, 0)$  with initial distance  $\text{grid}[0][0]$ , since the courier must traverse the starting cell. This is consistent with the coursework example, in which the optimal sum begins with  $\text{grid}[0][0] = 7$ .

A naive dynamic-programming recurrence over only top and left neighbours is **incorrect** here: because movement is permitted in all four directions, the state-transition graph contains cycles, and the optimal cost at cell  $(i, j)$  may depend on neighbours that are themselves reached *via*  $(i, j)$ . The problem is therefore not solvable by a one-pass DP on a DAG.

Since all edge weights are non-negative, **Dijkstra's algorithm** is both correct and asymptotically efficient. Using a binary min-heap as the priority queue yields a time complexity of  $O(mn \log(mn))$ . A 2-D `dist` array provides  $O(1)$  lookups, and stale heap entries are discarded lazily upon extraction.

#### 4.1.2 Python Implementation

```
1 import heapq
2 from typing import List
3
4 def min_delivery_time(grid: List[List[int]]) -> int:
5     """Return the minimum travel time using Dijkstra's algorithm."""
6     if not grid or not grid[0]:
7         return 0
8
9     m, n = len(grid), len(grid[0])
10
11     dist = [[float('inf')] * n for _ in range(m)]
12     dist[0][0] = grid[0][0]
13
14     heap = [(grid[0][0], 0, 0)]
15
16     # Four-directional moves: up, down, left, right
17     directions = ((-1, 0), (1, 0), (0, -1), (0, 1))
18
19     while heap:
20         d, i, j = heapq.heappop(heap)
21
22         if (i, j) == (m - 1, n - 1):
23             return d
24
25         if d > dist[i][j]:
26             continue
27
```

```

28     for di, dj in directions:
29         ni, nj = i + di, j + dj
30         if 0 <= ni < m and 0 <= nj < n:
31             new_dist = d + grid[ni][nj]
32             if new_dist < dist[ni][nj]:
33                 dist[ni][nj] = new_dist
34                 heapq.heappush(heap, (new_dist, ni, nj))
35
36     return dist[m - 1][n - 1]

```

Listing 6: Dijkstra-based minimum delivery time algorithm.

#### 4.1.3 Correctness and Efficiency

The algorithm maintains the invariant that whenever a cell  $(i, j)$  is extracted from the heap,  $\text{dist}[i][j]$  equals the true shortest-path distance from  $(0, 0)$  to  $(i, j)$ . This follows from the standard Dijkstra correctness argument: since all weights are non-negative, no path discovered later can improve a distance that is currently minimal in the heap.

Each vertex is extracted from the heap at most once with its final distance, and each of its at most four outgoing edges triggers at most one heap insertion. With  $|V| = mn$  and  $|E| = O(mn)$ , the total running time is  $O((|V| + |E|) \log |V|) = O(mn \log(mn))$ . Auxiliary space is  $O(mn)$  for the `dist` array and the heap.

#### 4.1.4 Verification on the Provided Example

For the input

$$\text{grid} = \begin{bmatrix} 7 & 2 & 7 & 7 \\ 5 & 5 & 4 & 2 \\ 9 & 1 & 1 & 2 \\ 5 & 6 & 7 & 2 \end{bmatrix},$$

the algorithm initialises  $\text{dist}[0][0] = 7$  and pushes  $(7, 0, 0)$  onto the heap. The following table traces the heap extractions, showing only relaxations that strictly improve a neighbour's distance.

| Step | Extracted $(d, i, j)$ | Neighbours relaxed (new dist)                | Heap (top entries)                     |
|------|-----------------------|----------------------------------------------|----------------------------------------|
| 1    | (7, 0, 0)             | $(0, 1) \leftarrow 9, (1, 0) \leftarrow 12$  | $[(9, 0, 1), (12, 1, 0)]$              |
| 2    | (9, 0, 1)             | $(0, 2) \leftarrow 16, (1, 1) \leftarrow 14$ | $[(12, 1, 0), (14, 1, 1), (16, 0, 2)]$ |
| 3    | (12, 1, 0)            | $(2, 0) \leftarrow 21$                       | $[(14, 1, 1), (16, 0, 2), (21, 2, 0)]$ |
| 4    | (14, 1, 1)            | $(2, 1) \leftarrow 15, (1, 2) \leftarrow 18$ | $[(15, 2, 1), (16, 0, 2), (18, 1, 2)]$ |
| 5    | (15, 2, 1)            | $(2, 2) \leftarrow 16, (3, 1) \leftarrow 21$ | $[(16, 0, 2), (16, 2, 2), (18, 1, 2)]$ |
| 6    | (16, 0, 2)            | $(0, 3) \leftarrow 23$                       | $[(16, 2, 2), (18, 1, 2), (23, 0, 3)]$ |
| 7    | (16, 2, 2)            | $(2, 3) \leftarrow 18$                       | $[(18, 1, 2), (18, 2, 3), (23, 0, 3)]$ |
| 8    | (18, 1, 2)            | $(1, 3) \leftarrow 20$                       | $[(18, 2, 3), (20, 1, 3), (23, 0, 3)]$ |
| 9    | (18, 2, 3)            | $(3, 3) \leftarrow 20$                       | $[(20, 1, 3), (20, 3, 3), (23, 0, 3)]$ |
| 10   | (20, 1, 3)            | no improvement                               | $[(20, 3, 3), (21, 2, 0), (23, 0, 3)]$ |
| 11   | (20, 3, 3)            | <b>destination extracted; return 20</b>      | —                                      |

The final `dist` matrix is

$$\text{dist} = \begin{bmatrix} 7 & 9 & 16 & 23 \\ 12 & 14 & 18 & 20 \\ 21 & 15 & 16 & 18 \\ 26 & 21 & 23 & 20 \end{bmatrix},$$

and the algorithm terminates at step 11 with  $\text{dist}[3][3] = 20$ , matching the expected output. This corresponds to the path

$$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (3,3)$$

with cumulative cost  $7 + 2 + 5 + 1 + 1 + 2 + 2 = 20$ .

Two observations are worth highlighting. First, the early-termination check fires the moment the destination is popped — there is no need to drain the rest of the heap, since any later extraction has distance at least 20. Second, step 8 illustrates why a naive top-to-bottom dynamic programming would fail:  $\text{dist}[1][3] = 20$  is established only via cell  $(1,2)$ , whose final distance 18 itself depends on  $(1,1)$ , and the optimal path to  $(3,3)$  then enters from  $(2,3)$  via the second column. Both observations confirm that four-directional movement requires Dijkstra rather than a one-pass DP.

## 4.2 Cell Count vs. Total Travel Time

**Yes — it is entirely possible for a minimum-time route to traverse strictly more cells than another route whose total travel time is strictly larger.**

### 4.2.1 Why the Two Objectives Differ

The cost of a path is  $\sum_{(i,j) \in \text{path}} \text{grid}[i][j]$ , *not* its cardinality. Because the objective minimises summed weight rather than hop count, a longer detour through low-cost cells can outperform a shorter path through high-cost cells. This is precisely why Dijkstra’s algorithm — which optimises cumulative weight — is required here, rather than BFS, which optimises the number of cells traversed and is correct only when all weights are equal.

### 4.2.2 Concrete Counterexample

Consider the following  $4 \times 5$  grid, in which row 2 and most of row 3 form a high-cost barrier:

$$\text{grid} = \begin{bmatrix} 1 & 100 & 1 & 1 & 1 \\ 1 & 1 & 1 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 1 \end{bmatrix}.$$

The courier starts at  $(0,0)$  and must reach  $(3,4)$ . We compare two routes:

| Route                         | Path                                                                                                                                                                      | Cells / Cost                                                         |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| $R_1$ (monotone, fewer cells) | $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4)$                                     | 8 cells, cost $1 + 100 + 1 + 1 + 1 + 100 + 100 + 1 = 305$            |
| $R_2$ (detour, more cells)    | $(0,0) \rightarrow (1,0) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (0,4) \rightarrow (1,4) \rightarrow (2,4) \rightarrow (3,4)$ | 10 cells, cost $1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 100 + 100 + 1 = 208$ |

Here  $R_2$  visits 10 cells with total cost 208, while  $R_1$  visits only 8 cells with total cost 305. The minimum-time route ( $R_2$ ) therefore contains *strictly more cells* ( $10 > 8$ ) than a higher-cost alternative ( $R_1$ ).

### 4.2.3 Justification

This counterexample formally refutes the intuition that minimum-time routes must be geometrically shortest. The asymmetry arises whenever the grid contains high-cost barriers separating

low-cost regions: a courier may be forced to circumnavigate the barrier, traversing many cheap cells rather than a few expensive ones. The Manhattan distance  $(m - 1) + (n - 1) + 1$  is a lower bound on the cell count of *monotone* paths (right/down only), but four-directional movement removes this constraint, permitting arbitrarily long detours when they are cost-effective.

This observation also confirms that BFS would be incorrect for the general problem: BFS minimises the number of edges traversed, which corresponds to minimising the cell count, not the total travel time. Conversely, in the special case where all cells have equal weight, the two objectives coincide, and BFS would yield a correct shortest-path algorithm.

### 4.3 Optimised Algorithm for $\{0,1\}$ Weights

When every cell has weight  $\text{grid}[i][j] \in \{0,1\}$ , where 0 denotes a fast road and 1 denotes a congested area, Dijkstra's algorithm is asymptotically suboptimal. The restricted weight set admits a specialised technique known as **0-1 BFS**, achieving a strict linear-time bound.

#### 4.3.1 Algorithmic Improvement: 0-1 BFS

**Key invariant.** In a 0-1 weighted graph, relaxing an edge either leaves the distance unchanged (weight 0) or increases it by exactly 1 (weight 1). Consequently, at any moment during the algorithm, the distances of vertices currently under consideration differ by *at most 1*. This monotone-bucketed structure can be maintained using a **double-ended queue (deque)** in place of a priority queue:

- When the algorithm relaxes a neighbour through a weight-0 edge (i.e., the neighbour cell has value 0), push the neighbour to the **front** of the deque, since its updated distance equals the current vertex's distance.
- When the algorithm relaxes a neighbour through a weight-1 edge (i.e., the neighbour cell has value 1), push the neighbour to the **back** of the deque, since its updated distance equals the current vertex's distance plus one.

The deque thus maintains the invariant that its contents are sorted in non-decreasing order of distance, with at most two distinct distance values present at any time. This emulates the behaviour of a min-heap priority queue at  $O(1)$  amortised cost per operation, eliminating the logarithmic factor incurred by Dijkstra. This approach generalises the standard BFS treatment — where a queue processes vertices in distance order for unweighted graphs — to the case of binary weights.

#### 4.3.2 Python Implementation

```

1 from collections import deque
2 from typing import List
3
4 def min_delivery_time_01(grid: List[List[int]]) -> int:
5     if not grid or not grid[0]:
6         return 0
7
8     m, n = len(grid), len(grid[0])
9
10    dist = [[float('inf')] * n for _ in range(m)]
11    dist[0][0] = grid[0][0]
12
13    dq = deque([(0, 0)])
14    directions = ((-1, 0), (1, 0), (0, -1), (0, 1))
15
16    while dq:

```

```

17     i, j = dq.popleft()
18
19     if (i, j) == (m - 1, n - 1):
20         return dist[i][j]
21
22     for di, dj in directions:
23         ni, nj = i + di, j + dj
24         if 0 <= ni < m and 0 <= nj < n:
25             w = grid[ni][nj]                # weight is 0 or 1
26             new_dist = dist[i][j] + w
27             if new_dist < dist[ni][nj]:
28                 dist[ni][nj] = new_dist
29                 if w == 0:
30                     dq.appendleft((ni, nj))    # zero-cost: front
31                 else:
32                     dq.append((ni, nj))        # unit-cost: back
33
34     return dist[m - 1][n - 1]

```

Listing 7: 0-1 BFS algorithm for binary-weight grids.

### 4.3.3 Complexity Analysis

**Time complexity:**  $O(mn)$ . A vertex is pushed into the deque only when its `dist` value strictly decreases. Since distances are non-negative integers bounded above by the path length  $mn$ , each vertex's `dist` is updated at most a constant number of times in the standard analysis (at most twice — once per incident weight class). Each push or pop on either end of the deque runs in  $O(1)$ . With  $|V| = mn$  and  $|E| = O(mn)$ , the total number of deque operations is  $O(|V| + |E|) = O(mn)$ , giving total time  $O(mn)$ .

**Space complexity:**  $O(mn)$ . Dominated by the `dist` matrix, which stores one value per cell. The deque holds at most  $O(mn)$  entries in the worst case. No other auxiliary structures of super-linear size are used.

### 4.3.4 Comparison with the General Algorithm

| Property         | General Dijkstra (Task 4-1) | 0-1 BFS (Task 4-3)         |
|------------------|-----------------------------|----------------------------|
| Time complexity  | $O(mn \log(mn))$            | $O(mn)$                    |
| Space complexity | $O(mn)$                     | $O(mn)$                    |
| Applicable to    | Any non-negative weights    | Weights in $\{0, 1\}$ only |
| Data structure   | Binary min-heap             | Double-ended queue         |

The 0-1 BFS achieves a strictly better time complexity by exploiting the restricted weight domain, eliminating the log factor associated with heap maintenance. This bound is asymptotically optimal, since any correct algorithm must inspect every cell in the worst case, requiring  $\Omega(mn)$  work.